RAIN: Region-Aware Inversion Network for Semantic Watermark Extraction

Zilai Li Independent Researcher

Abstract

Semantic watermarks for diffusion models embed ownership information into the generative process while preserving perceptual quality, but the watermark extraction process in some algorithms, like Gaussian Shading, conventionally requires multi-step diffusion inversion to recover the initial noise. Recent one-step methods show that this cost can be reduced substantially. We study this problem through extended rectified flow and conditional regression. The key observation is that, near the high-SNR image endpoint, recovering a useful noise statistic given by the first-step output of the extended rectified flow is much simpler than reconstructing the full inverse trajectory, and that is enough to extract the watermark, since extracting a watermark given by Gaussian Shading only requires knowing which Gaussian region the original latent occupies. Based on this observation, we propose a lightweight, prompt-free extractor to predict the noise statistics. Meanwhile, we further decompose endpoint statistic recovery into an image-like anchor and a noise-oriented residual, thereby increasing the model's ability to leverage GPU parallel computation. The resulting method avoids iterative inversion and repeated evaluation of a diffusion-scale U-Net, providing an efficient one-step extraction pipeline with a concise theoretical interpretation. For the Gaussian Shading watermark extraction process, the computational cost of our algorithm is 14.394 GFLOPs, which is 47 times fewer than FARI and OSI.

1 Introduction

Semantic watermarks for diffusion models [6, 8, 10, 23, 24, 25] embed ownership information into the generative process rather than modifying the final image after generation. Gaussian Shading (GS) [25] is a representative example: it encodes a secret message into the initial Gaussian noise while preserving its marginal Gaussian distribution. However, conventional GS watermark extraction requires multi-step DDIM inversion [15] to recover the noise from the generated image, and reducing the computation cost of DDIM inverse for watermark extraction is important. Although there exist algorithms aiming to improve DDIM inversion [21, 22], or reducing inference cost [3, 14, 19, 20, 27, 28, 31], neither of these algorithms is for watermark extraction. Recent one-step watermark extraction methods [5, 26] suggest that this expensive inverse process may be unnecessary.

Our first observation is that regression does not, in general, recover the particular noise that generated an observed image. If several noises are the starting points of the inference trajectory that ends at the same or similar image, an L_2 regression objective learns the mean

© 2026. The copyright of this document resides with its authors.
It may be distributed unchanged freely in print or electronic forms.

of the noise. And this situation is possible, since the Gaussian noise is in a vast space, while the image only occupies a low-dimensional manifold. This is closely related to the intuition behind Tweedie's formula [7]: under Gaussian corruption, when we sample a noisy image without knowing the original image, the best prediction, in the sense of minimizing the L2 distance between the prediction and the true image, is the mean over the possible images that can generate this noisy image via adding random noise. For Gaussian Shading, knowing the mean can be sufficient because watermark extraction only requires the predicted noise to preserve the encoded Gaussian regions. If different noises generate the same image, then DDIM inverse will produce a noise sample different from the original noise, but invalid watermark extraction is rare in normal DDIM inverse process. So we further assume that predicting the mean or median of these noises is also sufficient and does not invalidate the GS, which motivates us to propose an algorithm aiming to model the statistics of noise in a particular region to extract the corresponding watermark: region-aware inverse network.

Our second observation comes from Rectified Flow [13]: In the high SNR region, the RF-2 algorithm will study the velocity field point to the means of noise that generate the current noisy image. Explicitly, the interpolation between image and noise to get the noisy image in the training is non-causal, and that cannot be used in inference; the regression loss lets the model learn the conditional mean velocity of the possible trajectories passing through the current noisy image. Iteratively following this causal velocity field can still reproduce the desired image distribution. Therefore, predicting a conditional mean rather than a particular paired endpoint is already a standard mechanism of generative flow models.

This perspective makes one-step watermark extraction relate to an especially simple task that generates an image in the high-SNR regime of inference. Explicitly, for the RF-2 algorithm, in the high-SNR regime, most of the generation process has already been completed, and the network only needs to learn the mean of the velocity that can complete the remaining part of the transport, which is an easy task, and it's equivalent to predicting the mean of the possible noise to generate the final image, a useful statistic for watermark extraction. We therefore hypothesize that this high-SNR watermark extraction task can be handled by a substantially smaller network than a full diffusion denoiser.

Based on these observations, we propose an aggressive lightweight estimator that directly predicts recoverable noise statistics from an almost-clean image. Our method avoids iterative inversion and decomposes the endpoint regression into easier components, substantially reducing the computational cost of watermark extraction. The inference computation and parameter count are both lower than the latest extraction methods, such as OSI and FARI.

2 Background

2.1 From DDPM Corruption to Extended Flow Matching and Rectified Flow

The DDPM algorithm use a special diffusion process [9] to gradually perturbs the sample image via random noise through a Markov chain,

$$q(X_t \mid X_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, X_{t-1}, \beta_t I\right), \tag{1}$$

where $\{\beta_t\}$ is a predefined noise schedule. Although this process adds noise step by step, the renowned property of Gaussian transitions [9] allows us to sample the noisy state at any

timestep directly:

$$X_t = \sqrt{\bar{\alpha}_t}\, X_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{2}$$

with $\bar{\alpha}_t = \prod_{i=1}^{t}(1-\beta_i)$.

Equation (2) reveals a useful interpretation of diffusion training. Instead of viewing DDPM only as a sequence of stochastic perturbations, each noisy sample X_t can be viewed as a point on a trajectory given by interpolation between a clean endpoint X_0 and a Gaussian-noise endpoint ε. In the standard DDPM construction,

$$X_0 \sim p_{\text{data}}, \qquad \varepsilon \sim \mathcal{N}(0, I), \qquad X_0 \perp \varepsilon, \tag{3}$$

so the two endpoints are sampled independently.

This interpolation viewpoint naturally connects DDPM to Flow Matching and Rectified Flow. More generally, let

$$(X_0, \varepsilon) \sim \pi_{0,\varepsilon} \tag{4}$$

denote an *arbitrary coupling* between an image endpoint and a noise endpoint, and consider

$$X_t = a_t X_0 + b_t \varepsilon, \tag{5}$$

where a_t and b_t are differentiable schedules. In the standard rectifly flow, $a_t = 1-t, b_t = t$, the t is continuous and is between zero to one. For a clean-to-noise convention, one may take $a_0 \approx 1, b_0 \approx 0$ and $a_1 \approx 0, b_1 \approx 1$. The corresponding non-causal velocity is

$$U_t = \dot{a}_t X_0 + \dot{b}_t \varepsilon. \tag{6}$$

, in which $\dot{a}_t$ is the deviation of the b_t with respect to t, and so the $\dot{b}_t$.

Once both endpoints are known, Eqs. (5)–(6) define a valid trajectory. However, this trajectory is *non-causal as an inference rule*: U_t generally depends on the hidden pair (X_0, ε) and therefore cannot be determined from the current state X_t alone. Consequently, the sample-wise interpolation is not yet a closed ODE that can be integrated during inference.

Flow matching resolves this problem by learning a deterministic vector field from the observable state. Under the squared regression objective

$$\min_v \mathbb{E}\left[\|v(X_t, t) - U_t\|_2^2\right], \tag{7}$$

the population-optimal field [1] is:

$$v^*(x,t) = \mathbb{E}[U_t \mid X_t = x]. \tag{8}$$

Importantly, rectified flow does not require the noise and image to be independent. Standard DDPM is only one special case, obtained by choosing the diffusion schedule in Eq. (2) together with the independent coupling $X_0 \perp \varepsilon$. More generally, X_0 and ε may be deterministically coupled, produced by a teacher sampler[12].

2.2 Gaussian Shading

Gaussian Shading (GS) [25] embeds a watermark directly into the terminal Gaussian latent while preserving its marginal distribution. Let the latent have shape $c \times h \times w$, let l denote the

number of embedded bits per scalar in the high-dimensional Gaussian noise, and let f_c and f_{hw} denote the channel and spatial repetition factors. The effective payload length is

$$k = \left\lfloor \frac{lchw}{f_c f_{hw}^2} \right\rfloor bits. \tag{9}$$

The payload is first repeated according to f_c and f_{hw} and then encrypted with a stream cipher, producing an approximately uniform pseudorandom bit stream.

GS uses the encrypted bits to select equal-probability regions of the standard Gaussian distribution. For an l-bit symbol $i \in \{0,\dots,2^l-1\}$, the corresponding interval is

$$I_i = \left(\Phi^{-1}\left(\frac{i}{2^l}\right), \Phi^{-1}\left(\frac{i+1}{2^l}\right)\right], \tag{10}$$

where Φ is the standard Gaussian CDF. A latent scalar is sampled from the Gaussian distribution truncated to the selected interval. Because the encrypted symbols are approximately uniform and the intervals have equal Gaussian probability, marginalizing over the symbols recovers the original standard Gaussian prior. Thus the watermark changes the latent region from which each scalar is sampled without changing the marginal distribution seen by the diffusion model.

Extraction reverses this procedure. An estimated terminal latent $\hat{z}_T$ is mapped back to its Gaussian regions, the corresponding symbols are decrypted, and repeated watermark bits are aggregated by voting. For the common setting $l = 1$, the two regions are simply the negative and positive halves of the Gaussian distribution, so extraction reduces to recovering the sign of each latent coordinate. Therefore GS does not require exact reconstruction of the original terminal noise: it only requires the recovered latent to preserve sufficiently many of the correct Gaussian decision regions.

3 Approach

3.1 One-Step Recovery as Conditional Regression

Let X_0 denote the observed image-side latent, let ε denote the terminal Gaussian noise associated with the generated sample, and let D denote an optional image perturbation for adversarial attack. We write a general one-step estimator as

$$\hat{\varepsilon} = f(\theta, X_0, D), \tag{11}$$

where f denotes the complete estimator and θ denotes its learnable parameters.

Under squared endpoint regression,

$$\min_{\theta} \mathbb{E}\left[\|f(\theta, X_0, D) - \varepsilon\|_2^2\right], \tag{12}$$

For Eq 12, the optimal output is the corresponding mean of noise conditional on X_0. Thus, the estimator is not required to identify the noise that generates the image; it only needs to predict the conditional endpoint statistic supported by the observation. Using L_1 regression, the optimization target is a conditional median. Predicting only the mean or median doesn't matter, since if one image maps to multiple noises, then the DDIM inverse will sample a noise that is different from the original noise, but that almost never invalidates the GS. So we

assume predicting the mean or median of noise in a limited region also still does not invalidate the GS, and use experiments to test it: training a neural network that does not aim to recover the endpoint, but returns the statistics of samples in a special region, to test whether it can properly extract the GS watermark.

For Gaussian Shading, this statistical relaxation is sufficient whenever the predicted endpoint remains in the same watermark decision region. Let GSDec_K denote the complete GS decoder. Our requirement is

$$\mathrm{GSDec}_K(\hat{\varepsilon}) = \mathrm{GSDec}_K(\varepsilon), \tag{13}$$

rather than $\hat{\varepsilon} = \varepsilon$. This distinction is especially natural for the common $l = 1$ setting, where each scalar is decoded primarily through its sign and repeated coordinates are aggregated by voting.

3.2 High-SNR Endpoint Regression

The conditional-regression view follows directly from the extended flow-matching formulation in Sec. 2. For an arbitrary coupled image–noise pair, let

$$X_t = a_t X_0 + b_t \varepsilon \tag{14}$$

define a non-causal interpolation with velocity U_t used in the training. The corresponding causal field is

$$v^*(x,t) = \mathbb{E}[U_t \mid X_t = x] = \frac{\dot{a}_t}{a_t} x + (\dot{b}_t - \frac{b_t}{a_t})\mathbb{E}[\varepsilon | X_t = x]. \tag{15}$$

Previous research [12] already shows that, under the usual regularity assumptions, this causal velocity field induces the same marginal evolution as the non-causal interpolation. Importantly, the result does not require X_0 and ε to be independent. When we use the original diffusion model to generate the noise-image pair, studying the means of the velocity will provide a different trajectory, where the velocity in the high-SNR area will point to the means of possible noise that can generate this image.

Meanwhile, near the image endpoint, the input X_t is very close to the final result, and predicting the velocity to finish the remaining trajectory is easy. So the watermark extractor that doesn't aim to reconstruct the complete noise-to-image/image-to-noise trajectory to reduce the computation cost can have a natural and simple training target: learn the endpoint statistic mean, since the original RF training target, as Eq. 15 shows, is equivalent to predicting the mean of possible endpoint ε, and from the perspective of RF2, this is a simpler objective target than exact trajectory recovery. This observation motivates us to use a lightweight network rather than repeatedly evaluating a diffusion-scale denoiser. We can also further assume that predicting the median is feasible even when the training neural network is small. We choose L1 loss to predict the median because previous research shows that DDPM [9], which trains a denoiser and denoises noisy lantent/image step by step, is equivalent to the extended rectified flow[12], and the L1 loss in the denoising task has also been reported to provide better performance[29]. We call this model, which aims to recover the statistics of the sample in a limited region, the region-aware inversion network.

3.3 Endpoint Parameterization and Perturbation

Existing one-step estimators can also be written within Eq. (11). As one example, FARI uses the DDPM-style parameterization

$$\hat{\varepsilon} = a_T \widetilde{X}_0 + b_T \, \varepsilon_\theta(\widetilde{X}_0, 0), \tag{16}$$

where

$$a_T = \sqrt{\bar{\alpha}_T}, \qquad b_T = \sqrt{1-\bar{\alpha}_T}, \tag{17}$$

and X_0 correspond to the latent in the image side, $\widetilde{X}_0 = D(X_0)$.

Another common setting of $\hat{\varepsilon}$ is to directly predict the noise. Explicitly, it's:

$$\hat{\varepsilon} = \varepsilon_\theta(D(X_0)) \tag{18}$$

We find that both of these two functions can successfully extract the watermark even if we use a very small neural network to optimize the regression loss.

Eq. 16 is based on DDPM, and we find that we can utilize the GPU more efficiently when we use a heuristic formula based on DDIM, which will be discussed in the next subsection.

3.4 Anchor–Residual Reverse Estimator

Equation (11) allows the complete estimator to contain multiple learned components, and Equation (16) is just one of the special cases given by DDPM; we also provide a DDIM version. Our design is motivated by a simple optimization consideration: directly mapping an image-like latent concentrated near the low-dimensional manifold to a high-dimensional Gaussian noise vector requires a map between two very different types of representation. We therefore decompose the prediction so that the first network predicts an image-like target while the second network models the remaining noisy component. By decomposing the network into two parts, we can further utilize the GPU and augment parallel capabilities.

For each noisy image X_t, the original diffusion model G_θ can predict a clean image-like latent $X_0^*(t)$. The first prompt-free network predicts an image-like anchor,

$$\widehat{X}_0 = F_0(D(X_0)). \tag{19}$$

The output $\widehat{X}_0$ is a coarse clean representation, and its target is obtained from the original inference process. Explicitly, in each timestep t, the diffusion model can input a noisy image X_t and output a clean image $X_0^*(t)$. The training loss of F_0 is

$$\mathcal{L}_1 = \mathbb{E}\left[\mathrm{Dist}\left(F_0(D(X_0)), X_0^*(t)\right)\right], \tag{20}$$

, and the *Dist* can be $L2$ distance or $L1$ distance. In the training, we fix $t = 0.9$, and *Dist* to be $L1$ distance.

The second network predicts an intermediate representation conditioned on the observed latent,

$$\widehat{X}_t = F_1(D(X_0)), \tag{21}$$

A lightweight head predicts its effective noise level,

$$\widehat{t} = F_2(\widehat{X}_t). \tag{22}$$

Using the DDIM parameterization, the terminal noise estimate is reconstructed algebraically as

$$\hat{\varepsilon} = f(\theta, X_0, D) = \sqrt{1-\bar{\alpha}_T}\,\frac{\widehat{X}_t - \sqrt{\bar{\alpha}_{\widehat{t}}}\,SG(\widehat{X}_0)}{\sqrt{1-\bar{\alpha}_{\widehat{t}}}} + \sqrt{\bar{\alpha}_T}\,SG(\widehat{X}_0). \tag{23}$$

where $SG(\cdot)$ denotes stop gradient.

The second recovery loss directly supervises the final endpoint:

$$\mathcal{L}_2 = \mathbb{E}\left[\mathrm{Dist}\left(\hat{\varepsilon}, \varepsilon\right)\right]. \tag{24}$$

Rather than fixing the intermediate noise level given by t, we allow the model to learn an effective $\widehat{t}$. During training, the timestep head detecting the noisy level of the output from F_1 is supervised on constructed interpolation states:

$$X_t^* = \sqrt{\bar{\alpha}_t}\,X_0^*(0.9) + \sqrt{1-\bar{\alpha}_t}\,\varepsilon, \tag{25}$$

with

$$\mathcal{L}_3 = \mathbb{E}\left[\mathrm{Dist}\left(F_2(X_t^*), t\right)\right]. \tag{26}$$

The complete training objective is

$$\mathcal{L}_{\mathrm{rev}} = \lambda_0\mathcal{L}_1 + \lambda_T\mathcal{L}_2 + \lambda_{\mathrm{reg}}\mathcal{L}_3. \tag{27}$$

By utilizing this special structure, F_0 and F_1 can be inferred in parallel, and the GPU can be utilized more efficiently. But it requires a small neural network to predict the *noisy level of the* $\widehat{X}_t$, the timestep head introduces only a small additional cost.

Our implementation uses standard NAFNet [4] backbones. No text encoder, prompt embedding, or cross-attention is required during extraction, and the source diffusion U-Net is absent from the learned reverse path.

Adversarial distortion training. Robustness is handled through the perturbation argument D in Eq. (11). For each training sample, we construct a candidate set $\mathcal{D}_K$ containing common image corruptions such as JPEG compression, resizing, cropping, masking, blur, noise, and brightness changes. We select the corruption that maximizes the current reverse-recovery loss,

$$d^* = \arg\max_{d\in\mathcal{D}_K} \mathcal{L}_{\mathrm{rev}}\left(f(\theta, X_0, d)\right), \tag{28}$$

and update the estimator using the selected example. This finite-set min–max procedure improves robustness without changing the underlying endpoint objective. The complete pipeline is shown in Fig. 1.

4 Experiments

4.1 Ablation experiment

4.1.1 Current experimental setting

To train our algorithm, the training set contains 89600 image-noise pairs generated offline. During training, there is no additional cost to involve the original diffusion model; all necessary data is already generated and stored in a reverse-distillation LMDB feature store.

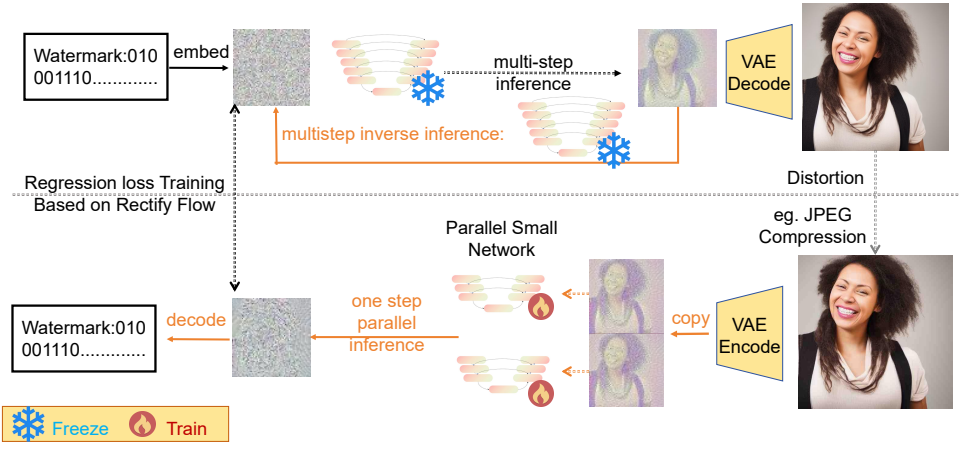


Figure 1: Extraction pipeline used in our implementation.

We train six extractors to do the ablation study, and all of them are prompt-free and use width-64 NAFNet [4] backbones with coordinatewise L_1 endpoint supervision.

The principal model is a two-branch reverse estimator given by Section 3.4. Its F_0 branch and F_1 branch are both standard NAFNet[4], each use two encoder stages with $[5,5]$ blocks, 12 middle blocks, and two decoder stages with $[5,5]$ blocks. The second branch additionally predicts an effective timestep and reconstructs terminal noise using Eq. (23). We compare it with two single-network baselines, each using the larger $[10,10]/24/[10,10]$ NAFNet architecture: DIRECT-x_T directly regresses terminal noise, while FARI LIKE predicts epsilon and converts it to x_T with the terminal DDPM coefficients. The objective target of the first network is described in Eq. 18, and the objective target of the second network is described by Eq. 16. All reported learned-model checkpoints correspond to 15,000 training iterations; the learning rate of the training is 1e-3; the optimizer is AdamW with a cosine annealing LR scheduler. The training set contains 89600 image-noise pairs. Meanwhile, all of the loss weights in Eq. 27 are 1.0. The captions used to generate the offline image-noise training pairs are sampled from LAION [17].

To evaluate watermarks extraction, the text-to-image generation at 512×512 resolution use Stable Diffusion v1.5 [16] and COCO captions [11]. Gaussian Shading's setting uses $f_c = 1$, $f_{hw} = 8$, $l = 1$, assume 10^6 users, and a target false-positive rate is 10^{-6}. Images are generated with the Diffusers UniPC scheduler [30] for 15 steps at guidance scale 5.5. For every caption, we use a watermarked noise to generate an image; we extract the watermark via: 1. choose Gaussian noise and generate image, 2. β-VAE [2] decode latent, 3. β-VAE encode image, 4. One-step extraction without caption.

4.1.2 Metrics

We follow the official Gaussian Shading [25] evaluation protocol and report bit accuracy, detection rate, and traceability rate. For $l = 1$, the recovered latent is converted to binary regions according to its sign and decoded using the official GS procedure. Detection and traceability are determined by the official thresholds τ_{onebit} and τ_{bits}, respectively. We use $f_c = 1$, $f_{hw} = 8$, a target false-positive rate of 10^{-6}, and 10^6 candidate users.

Table 1: SD v1.5 extractor ablation under the matched ten-condition image-distortion protocol on 1,000 COCO captions. All six 15,000-iteration checkpoints use the same cached images, GS states, VAE pathway, and deterministic attack realizations.

Bit accuracy (%) ↑

Model	Clean	JPEG	Crop/ mask	Drop	Resize	G. blur	Median	G. noise	S&P	Bright.
No-Adv Fari-Like	99.99%	97.70%	93.67%	93.33%	98.50%	94.65%	99.04%	98.39%	91.31%	95.39%
No-Adv Direct-X_T	99.99%	97.73%	93.70%	93.25%	98.51%	94.63%	99.06%	98.41%	91.33%	95.34%
No-Adv Two brach	99.99%	97.63%	91.59%	90.82%	98.42%	94.38%	98.99%	98.38%	91.28%	93.02%
Adv Fari-Like	100.00%	98.31%	94.51%	94.50%	99.03%	97.65%	99.26%	98.78%	93.81%	96.43%
Adv Direct-X_T	99.99%	98.32%	94.51%	94.49%	99.04%	97.68%	99.26%	98.79%	93.87%	96.47%
Adv Two brach	99.99%	98.15%	93.72%	93.64%	98.93%	97.24%	99.17%	98.67%	93.12%	96.09%

Detection rate (%) ↑

Model	Clean	JPEG	Crop/ mask	Drop	Resize	G. blur	Median	G. noise	S&P	Bright.
No-Adv Fari-Like	100.00%	99.70%	100.00%	100.00%	99.90%	100.00%	100.00%	99.90%	98.70%	97.60%
No-Adv Direct-X_T	100.00%	99.70%	100.00%	100.00%	100.00%	100.00%	100.00%	99.90%	98.80%	97.50%
No-Adv Two brach	100.00%	99.60%	100.00%	100.00%	100.00%	99.90%	100.00%	99.90%	98.50%	95.10%
AdvFari-Like	100.00%	99.80%	100.00%	100.00%	100.00%	100.00%	100.00%	99.90%	99.20%	97.80%
Adv Direct-X_T	100.00%	99.80%	100.00%	100.00%	100.00%	100.00%	100.00%	99.90%	99.40%	98.10%
Adv Two brach	100.00%	99.70%	100.00%	100.00%	100.00%	100.00%	100.00%	99.90%	99.20%	97.80%

Traceability rate (%) ↑

Model	Clean	JPEG	Crop/ mask	Drop	Resize	G. blur	Median	G. noise	S&P	Bright.
No-Adv Fari-Like	100.00%	99.40%	100.00%	100.00%	99.70%	99.70%	100.00%	99.50%	95.70%	95.80%
No-Adv Direct-X_T	100.00%	99.40%	100.00%	100.00%	99.80%	99.70%	100.00%	99.60%	95.80%	95.90%
No-Adv Two brach	100.00%	99.20%	100.00%	99.90%	99.70%	99.60%	100.00%	99.70%	96.10%	92.00%
Adv Fari-Like	100.00%	99.40%	100.00%	100.00%	99.80%	99.80%	100.00%	99.70%	98.60%	97.10%
Adv Direct-X_T	100.00%	99.50%	100.00%	100.00%	99.90%	99.90%	100.00%	99.80%	98.30%	97.40%
Adv Two brach	100.00%	99.10%	100.00%	100.00%	99.80%	99.70%	100.00%	99.70%	97.90%	96.50%

4.1.3 Ablation results

We evaluate all six combinations on the same 1,000 cached COCO samples: direct X_T prediction, Fari-Like Prediction, and DDIM heuristic prediction, with or without adversarial training. For every condition, all extractors receive the same attacked image and VAE-encoded latent. Table 1 shows that adversarial training consistently improves robustness under severe distortions while preserving clean extraction accuracy. The direct-x_T and epsilon parameterizations remain closely matched, whereas the two-branch model is more sensitive without distortion training and recovers most of the gap after adversarial training.

Table 2 reports extractor-only wall-clock measurements on 100 clean cached images. This experiment deploys the neural networks on two RTX PRO 6000 GPUs. We use FP32 inference, batch size 10, and three warm-up batches on the same two-GPU system. The single-NAFNet models use data parallelism, splitting each batch equally across the two devices, while the two-branch model places the x_0 and x_T branches on separate devices. Host-to-device input transfer, VAE encoding, and GS decoding are excluded; cross-device output gathering and terminal-noise reconstruction are included. The two-branch allocation requires 2.993–3.085 ms per image, compared with 4.847–4.977 ms for the single-network alternatives. Adversarial training changes the weights but not the architecture and has little effect on extraction latency.

Table 2: Latency for extracting 100 watermarks with the extractor deployed on two RTX PRO 6000 GPUs. Single-network models use data parallelism; the two-branch model uses branch parallelism. Lower latency and higher throughput are better.

Method	Total (ms) ↓	ms/image ↓	images/s ↑	GFLOPs	GMACs	params
No-adv Direct x_T	489.258	4.893	204.39	**13.833**	**6.873**	**14,539,460**
No-adv Fari-like	484.709	4.847	206.31	**13.833**	**6.873**	**14,539,460**
No-adv Two-branch	**299.330**	**2.993**	**334.08**	14.394	7.148	15,467,792
adv Direct x_T	485.419	4.854	206.01	**13.833**	**6.873**	**14,539,460**
adv Fari-like	497.652	4.977	200.94	**13.833**	**6.873**	**14,539,460**
adv Two-branch	**308.505**	**3.085**	**324.14**	14.394	7.148	15,467,792

Table 3: Matched SD v2.1 robustness comparison under the FARI-style image-distortion protocol on 1,000 COCO captions. The first group reports bit accuracy on watermarked images. The second group reports the *maximum* bit accuracy when we compare the cached watermark with the extracted bit strings given by the *controlled* images group generated from the same prompts but with different noise and *no watermark*.

	Mean bit accuracy (watermarked) ↑				Max bit accuracy (controlled) ↑			
Distortion	FARI	OSI	Ours	DDIM inv (50)	FARI	OSI	Ours	DDIM inv (50)
Clean VAE roundtrip	99.99%	100.00%	99.99%	100.00%	64.45%	65.63%	64.84%	64.06%
JPEG (Q=25)	98.33%	99.09%	97.81%	97.63%	64.45%	64.84%	65.62%	64.45%
Random crop/mask (0.6)	95.63%	97.67%	93.65%	95.52%	64.84%	64.45%	64.06%	64.84%
Random drop (0.8)	95.09%	97.56%	93.49%	94.93%	64.45%	64.06%	64.84%	64.06%
Resize (0.25)	99.09%	99.48%	98.65%	98.59%	65.23%	64.84%	65.23%	64.45%
Gaussian blur (r=4)	97.55%	98.99%	96.61%	95.18%	64.84%	64.84%	65.23%	65.63%
Median blur (k=7)	99.22%	99.67%	98.93%	99.03%	64.84%	64.06%	64.45%	65.63%
Gaussian noise ($\sigma = 0.05$)	98.79%	99.52%	98.47%	98.62%	64.84%	64.45%	64.84%	65.63%
Salt-and-pepper (p=0.05)	94.14%	99.99%	92.42%	91.17%	65.63%	66.02%	63.67%	66.41%
Brightness (factor=6)	95.97%	96.62%	95.48%	95.20%	65.63%	64.45%	64.06%	64.45%

4.2 Comparison with SOTA extraction

4.2.1 Robustness comparison protocol

For the matched SOTA comparison, we use Stable Diffusion v2.1-base for both generation and extraction. We evaluate the DDIM inverse 50-step, the official FARI, the official OSI checkpoint and our adversarially trained SD v2.1 two-branch checkpoint on the same 1,000 COCO captions, seeds, GS keys, 512×512 images, 15-step UniPC schedule, and VAE re-encoding pathway. Each generated image is independently evaluated under all ten distortions given by FARI[26]: the unmodified VAE decode-encode roundtrip; JPEG quality 25; crop/mask ratio 0.6; random-drop ratio 0.8; resize ratio 0.25; Gaussian blur radius 4; median-filter kernel 7; additive Gaussian noise with standard deviation 0.05 on the $[0, 1]$ image scale; salt-and-pepper probability 0.05; and brightness factor 6. Attack randomness is deterministically seeded per caption and condition. Table 3 reports mean GS bit accuracy, detection rate, and traceability rate. Meanwhile, to ensure the algorithm doesn't memorize, we also use random Gaussian noise that hasn't been augmented with GS to generate images, and extract the corresponding noise, report the Max bit accuracy given by comparing those extracted noise with all cached noise with watermark.

Table 4 compares the backbone cost of one latent-to-noise watermark-extraction pass. Our

Table 4: Backbone cost of one latent-to-noise watermark extraction, measured with `calflops`; lower is better. Image encoding and GS decoding are excluded for all methods.

Method	GFLOPs ↓	GMACs ↓	Params (M) ↓	latency (ms/image)↓
FARI	682.675	341.079	868.469	37.344
OSI	678.723	339.103	865.911	31.320
Ours	**14.394**	**7.148**	**15.468**	**1.392**

Table 5: The number of false positive on *controlled* images with *no watermark*. For each distortion, we run extraction, recover a bit string, and declare a detection if it passes a threshold after comparing it with cached watermarks. We do 1,000,000 comparisons and, based on the max bit accuracy, determine the detection result;

Extractor	**Clean**	**JPEG**	**Crop/mask**	**Drop**	**Resize**	**GBlur**	**MBlur**	**GNoise**	**S&P**	**Bright**
FARI	0	0	1	0	3	1	1	3	1	1
OSI	3	2	0	0	1	2	0	0	2	0
Ours	1	3	0	1	1	1	0	1	0	0
DDIM inv (50)	0	0	1	0	0	2	1	1	2	0

lightweight two-branch extractor requires only 14.394 GFLOPs and 7.148 GMACs, reducing computation by approximately 47.4× relative to FARI and 47.2× relative to OSI in FLOPs. It also uses only 15.468M total parameters, compared with 868.469M for FARI and 865.911M for OSI. FARI trains only 2.558M LoRA parameters, but extraction still executes and stores the frozen SD v2.1 U-Net. For a matched backbone comparison, image generation, latent encoding (including OSI's encoder and quantizer), and GS decoding are excluded. We also report the latency of watermark extraction on 224 images using 2 A800 GPUs, with FARI and OSI's batch size set to 2 while our method's batch size is 112. By using this batch size, the peak GPU allocation of FARI is 3518/3510 MiB, the peak GPU allocation of OSI is 3498/3489 MiB, and the peak GPU allocation of ours is 1098/1816 MiB. The extraction latency of FARI and OSI is 37.34 and 31.32 ms/image, while our is 1.39 ms/image.

We also report the number of false positives in Table 5. Simultaneously, we report the watermark diversity there to ensure a fair comparison: the number of keys, nonces, and messages we use in the GS is 1000, and all of them are unique. And the random seed to initiated training is different from each seed use to get the initiate Gaussian noise.

5 Conclusion

Gaussian Shading does not require a detector to reconstruct a unique floating-point terminal latent. Its cryptographic sampler partitions the Gaussian prior into finitely many equal-probability regions, and the final decoder only requires enough of those region labels to survive decryption and repetition voting. Extended flow matching and DDPM-style ε regression make the complementary statistical point: regression from a partially observed state naturally learns conditional statistics of hidden endpoints, not sample-specific hidden pairs. These two facts together motivate direct one-step watermark extraction.

We therefore formulate GS extraction as decision-region recovery and propose a prompt-free two-NAFNet reverse estimator trained with intentional L_1 loss and finite-set adversarial image distortions. The two-branch anchor/interpolation parameterization is an optimization

method rather than a claim that a new continuous trajectory has been derived.

6 Further Discussion

Revisiting the trajectory interpretation of one-step extraction. FARI uses a simple regression loss, which is different from the SOTA distillation loss [14, 18, 27, 28] that tends to generate a sample from the target distribution directly in one step. The objective target given by FARI is the one equivalent to the extended RF2 algorithm in the high SNR stage [12]. And whether it can generate a sample in one step is determined not by the trajectory characteristics of the extended RF-1 algorithm, but by the trajectory characteristics of the extended RF-2 algorithm. FARI's trajectory interpretation effectively treats watermark extraction as a sample-generation problem, but this interpretation is questionable.

Latest DMD2 algorithm remove regression loss for the sake of the image generation. Another evidence that the regression loss would not let the model directly generate a real sample from the target distribution is the DMD2 [27] algorithm which remove the regression loss since it violate the distribution matching target.

References

[1] Quentin Bertrand, Anne Gagneux, Mathurin Massias, and Rémi Emonet. On the closed-form of flow matching: Generalization does not arise from target stochasticity. *arXiv preprint arXiv:2506.03719*, 2025.

[2] Christopher P Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nick Watters, Guillaume Desjardins, and Alexander Lerchner. Understanding disentangling in backslash beta-vae. *arXiv preprint arXiv:1804.03599*, 2, 2018.

[3] Defang Chen, Zhenyu Zhou, Can Wang, Chunhua Shen, and Siwei Lyu. On the trajectory regularity of ode-based diffusion sampling. *arXiv preprint arXiv:2405.11326*, 2024.

[4] Liangyu Chen, Xiaojie Chu, Xiangyu Zhang, and Jian Sun. Simple baselines for image restoration. In *European Conference on Computer Vision*, 2022.

[5] Yuwei Chen, Zhenliang He, Jia Tang, Meina Kan, and Shiguang Shan. Osi: One-step inversion excels in extracting diffusion watermarks. *arXiv preprint arXiv:2602.09494*, 2026.

[6] Hai Ci, Pei Yang, Yiren Song, and Mike Zheng Shou. Ringid: Rethinking tree-ring watermarking for enhanced multi-key identification. In *European conference on computer vision*, pages 338–354. Springer, 2024.

[7] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[8] Samuel Gunn, Xuandong Zhao, and Dawn Song. An undetectable watermark for generative image models. In *International Conference on Learning Representations*, volume 2025, pages 6612–6637, 2025.

[9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

[10] Huayang Huang, Yu Wu, and Qian Wang. Robin: Robust and invisible watermarks for diffusion models with adversarial optimization. *Advances in Neural Information Processing Systems*, 37:3937–3963, 2024.

[11] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014.

[12] Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

[13] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[14] S Luo, Y Tan, S Patil, D Gu, P Von Platen, A Passos, L Huang, J Li, and H Zhao. Lcm-lora: A universal stable-diffusion acceleration module. arxiv 2023. *arXiv preprint arXiv:2311.05556*.

[15] Ron Mokady, Amir Hertz, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Null-text inversion for editing real images using guided diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6038–6047, 2023.

[16] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022.

[17] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in neural information processing systems*, 35:25278–25294, 2022.

[18] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. *arXiv preprint arXiv:2310.14189*, 2023.

[19] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. 2023.

[20] Vinh Tong, Trung-Dung Hoang, Anji Liu, Guy Van den Broeck, and Mathias Niepert. Learning to discretize denoising diffusion odes. In *International Conference on Learning Representations*, volume 2025, pages 47244–47282, 2025.

[21] Bram Wallace, Akash Gokul, and Nikhil Naik. Edict: Exact diffusion inversion via coupled transformations. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22532–22541. IEEE, 2023.

[22] Fangyikang Wang, Hubery Yin, Yuejiang Dong, Huminhao Zhu, Chao Zhang, Hanbin Zhao, Hui Qian, and Chen Li. Belm: Bidirectional explicit linear multi-step sampler for exact inversion in diffusion models. *Advances in Neural Information Processing Systems*, 37:46118–46159, 2024.

[23] Yuxin Wen, John Kirchenbauer, Jonas Geiping, and Tom Goldstein. Tree-rings watermarks: Invisible fingerprints for diffusion images. *Advances in Neural Information Processing Systems*, 36:58047–58063, 2023.

[24] Jindong Yang, Han Fang, Weiming Zhang, Nenghai Yu, and Kejiang Chen. T2smark: balancing robustness and diversity in noise-as-watermark for diffusion models. *Advances in Neural Information Processing Systems*, 38:118642–118668, 2026.

[25] Zijin Yang, Kai Zeng, Kejiang Chen, Han Fang, Weiming Zhang, and Nenghai Yu. Gaussian shading: Provable performance-lossless image watermarking for diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12162–12171, 2024.

[26] Yang et al. Fari: Official implementation. https://github.com/0xD009/FARI, 2026. Accessed September 2026.

[27] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and Bill Freeman. Improved distribution matching distillation for fast image synthesis. *Advances in neural information processing systems*, 37:47455–47487, 2024.

[28] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6613–6623, 2024.

[29] Hang Zhao, Orazio Gallo, Iuri Frosio, and Jan Kautz. Loss functions for image restoration with neural networks. *IEEE Transactions on computational imaging*, 3(1):47–57, 2016.

[30] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. Unipc: A unified predictor-corrector framework for fast sampling of diffusion models. *Advances in Neural Information Processing Systems*, 36:49842–49869, 2023.

[31] Zhenyu Zhou, Defang Chen, Can Wang, and Chun Chen. Fast ode-based sampling for diffusion models in around 5 steps. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7777–7786, 2024.